

Sri Lanka Institute of Information Technology

Faculty of Computing

Dr.Sanvitha Kasthuriarachchi

IT2130 - Operating Systems and System Administration

Year 02 and Semester 02

Lecture 07

Process Synchronization

Outline

- Introduction to Synchronization
- The Critical-Section Problem
- Semaphores
- Classical Problems of Synchronization
- Problems with Semaphores
- Monitors

Background

- Processes are executing concurrently while interrupting the executions of other processes.
- The concurrent access to shared data may result in data inconsistencies.
- Maintaining data consistency requires mechanisms to ensure the orderly execution of cooperating processes.

Data Inconsistency due to Concurrency

- Illustration of the problem:

Suppose that we wanted to provide a solution to the consumer-producer problem that fills all the buffers. We can do so by having an integer counter that keeps track of the number of full buffers. Initially, counter is set to 0. It is incremented by the producer after it produces a new buffer and is decremented by the consumer after it consumes a buffer.

Producer – Consumer Illustration

Producer

```
while (true)
{

    while (counter == BUFFER_SIZE) ;
        /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
    counter++;
}
```

counter++ could be implemented as

```
register1 = counter
register1 = register1 + 1
counter = register1
```

Consumer

```
while (true)
{

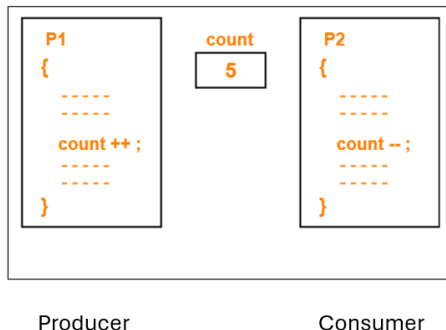
    while (counter == 0) ;
        /* do nothing */
    next_consumed = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    counter--;
}
```

counter-- could be implemented as

```
register2 = counter
register2 = register2 - 1
counter = register2
```

Producer – Consumer Illustration

- Assume the initial value of count is 5 and producer starts the execution. The producer increments count in its local copy and before the count is updated, the consumer starts. Then the consumer reads an incorrect value to proceed.
- This can be happened in the other way as well. Then the count may have a different value.



Race Condition

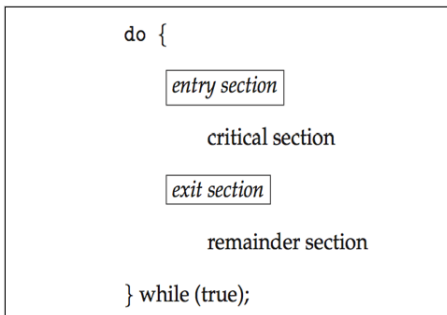
- This is a situation where several processes access and manipulate the shared data concurrently. Then the outcome of the execution depends on the order in which the access takes place.
- In order to prevent the race condition, it must ensure that only one process at a time can be manipulating the shared data.
- Then the process synchronization applies.

Critical Section of a Process

- The critical section of a process is a segment of code which the shared data is getting changed.
- The shared data includes common variables, shared buffers shared tables, shared files etc.

Critical Section Problem

- Each process must ask permission to enter critical section in **entry section**, may follow critical section with **exit section**, then **remainder section**
- General structure of process P_1



Critical Section Problem

- Consider a system with n processes $\{P_0, P_1, \dots, P_{n-1}\}$, where each process has a critical section.
- If more than one process executes its critical section at the same time, they may concurrently access and modify shared data. This situation is known as the **critical section problem**.
- If the critical section problem is not handled properly, it can lead to data inconsistency.

Solution to Critical-Section Problem

- 1 **Mutual Exclusion** – If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
- 2 **Progress** – If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely.
- 3 **Bounded Waiting** – A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.

How the Critical Section Problem can be Avoided

Simplest solution:

Each process disables all interrupts just after entering its critical section and re-enables them just before leaving it. However, enabling/disabling interrupts is a privileged instruction.

Solution in kernel mode:

- **Preemptive kernel:** a process can be preempted while running in kernel mode. More responsive and suitable for real-time systems.
- Otherwise, the kernel is non-preemptive kernel: allows the process to run until it exits kernel mode, blocks, or voluntarily yields CPU. Non-preemptive kernel is free from race conditions.

Solutions for Critical Section Problem

- There are three software-based solutions and two hardware-based solutions.

Software based solutions	Hardware based solutions
Algorithm 1 - for two processes	Test and Set Solution
Algorithm 2 - for two processes	Swap Solution
Algorithm 3 - Peterson's solution (for two processes): Combination of Algorithms 1 and 2	
Bakery Algorithm – for multiple processes	

ALGORITHM 1: Software-based Solution for two processes

var turn: (i, j); // Initially turn = i; turn = i means P_i can enter its CS

Process P_i

repeat

while $turn \neq i$ **do** no-op;

Critical Section

$turn = j$;

Remainder Section

until false;

Process P_j

repeat

while $turn \neq j$ **do** no-op;

Critical Section

$turn = i$;

Remainder Section

until false;

The Busy waiting condition occurs

- *Busy waiting: continuously testing a variable waiting for some value to appear*

ALGORITHM 2: Software-based Solution for two processes

Initially $flag[i] = flag[j] = false$; $flag[i] = true$ means P_i wants to enter its CS

Process P_i

repeat

$flag[i] = true$;

while $flag[j]$ **do** *no-op*;

 Critical Section

$flag[i] = false$;

 Remainder Section

until *false*;

Process P_j

repeat

$flag[j] = true$;

while $flag[i]$ **do** *no-op*;

 Critical Section

$flag[j] = false$;

 Remainder Section

until *false*;

ALGORITHM 3: Software-based Solution for two processes

Peterson's solution: Combine shared variables of Algorithms 1 and 2

Process P_i

repeat

$flag[i] = true;$

$turn = j;$

while $flag[j]$ **and** $turn = j$ **do no-op;**

 Critical Section

$flag[i] = false;$

 Remainder Section

until $false;$

Process P_j

repeat

$flag[j] = true;$

$turn = i;$

while $flag[i]$ **and** $turn = i$ **do no-op;**

 Critical Section

$flag[j] = false;$

 Remainder Section

until $false;$

Bakery Algorithm – Software solution for n processes

Process P_i

repeat

choosing[i] = *true*;

number[i] = $\max(\text{number}[0], \text{number}[1], \dots, \text{number}[n-1]) + 1$;

choosing[i] = *false*;

for $j = 0$ **to** $n-1$ **do**

begin

while *choosing*[j] **do** no-op;

while *number*[j] $\neq 0$ **and** (*number*[j], j) < (*number*[i], i) **do** no-op;

end;

critical section

number[i] = 0;

remainder section

until *false*

- Before entering the critical section, each process receives a number.
- The holder of the smallest number enters the critical section.

Synchronization Hardware

- Use special hardware instructions.
 - Test-and-set
 - Swap
- These are atomic instruction - They cannot be interrupted until it completes its execution

// Test and set the content of a word
atomically

```
function Test-and-Set (var boolean: target)
begin
    Test-and-Set = target;
    target = true;
end;
```

// Swapping instruction *atomically*

```
procedure Swap (var boolean: a, b)
var boolean: temp;
begin
    temp = a;
    a = b;
    b = temp;
end;
```

Synchronization Hardware

Test-and-Set

var boolean: lock; lock is a shared variable, initially set to false.

Repeat // Process P_i

while *Test-and-Set (lock)* **do** *no-op*;

 Critical Section

lock = false;

 Remainder Section

until *false*;

Repeat // Process P_j

while *Test-and-Set (lock)* **do** *no-op*;

 Critical Section

lock = false;

 Remainder Section

until *false*;

Synchronization Hardware

Swap

Repeat // Process P_i
 $key = true;$
 repeat
 $Swap(lock, key);$
 until $key = false;$
 Critical section
 $lock = false;$
 Remainder section
until $false;$

Repeat // Process P_j
 $key = true;$
 repeat
 $Swap(lock, key);$
 until $key = false;$
 Critical section
 $lock = false;$
 Remainder section
until $false;$

Semaphore

- This is a synchronization tool that provides more sophisticated way for process to synchronize their activities.
- Semaphore is an integer variable (S)
- Can only be accessed via two indivisible (atomic) operations
 - wait()
 - signal()

- Definition of the wait() operation

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

- Definition of the signal() operation

```
signal(S) {  
    S++;  
}
```

Semaphore Usage

- Counting semaphore – integer value can range over an unrestricted domain
- Binary semaphore – integer value can range only between 0 and 1 (Same as a mutex lock)
- Consider **P1** and **P2** that require **S1** to happen before **S2**
- Create a semaphore “synch” initialized to 0

P1:

```
S1;  
signal(synch);
```

P2:

```
wait(synch);  
S2;
```

- Can implement a counting semaphore S as a binary semaphore

Semaphore Implementation

- Must guarantee that no two processes can execute the `wait()` and `signal()` on the same semaphore at the same time.
- Semaphores are implemented in two methods.
 - Spinlock – uses busy waiting
 - Block and wakeup - block the execution and wait in a semaphore list. wait.

Classical Problems of Synchronization

- Classical problems used to test newly-proposed synchronization schemes:
 - Bounded-Buffer Problem
 - Readers and Writers Problem
 - Dining-Philosophers Problem

Bounded-Buffer Problem

- n buffers, each can hold one item.
- Semaphore `mutex` initialized to the value 1.
- Semaphore `full` initialized to the value 0.
- Semaphore `empty` initialized to the value n .

Bounded Buffer Problem (Cont.)

- The structure of the producer process

```
do {  
    ...  
    /* produce an item in next_produced */  
    ...  
    wait(empty);  
    wait(mutex);  
    ...  
    /* add next produced to the buffer */  
    ...  
    signal(mutex);  
    signal(full);  
} while (true);
```

Bounded Buffer Problem (Cont.)

- The structure of the consumer process

```
Do {  
    wait(full);  
    wait(mutex);  
  
    ...  
    /* remove an item from buffer to next_consumed */  
    ...  
    signal(mutex);  
    signal(empty);  
  
    ...  
    /* consume the item in next consumed */  
    ...  
} while (true);
```

Readers-Writers Problem

- Multiple readers want to read shared data and multiple writers want to write (modify) the same shared data.
 - Many readers can read at the same time.
 - Only one writer can write at a time.
 - When a writer is writing, no reader can read.
 - Two writers cannot write at the same time.

Readers-Writers Problem (Cont.)

- Shared Data
 - Data set
 - Semaphore **rw_mutex** initialized to 1
 - Semaphore **mutex** initialized to 1
 - Integer **read_count** initialized to 0
- The structure of a writer process

```
do {  
    wait(rw_mutex);  
  
    /* writing is performed */  
    ...  
    signal(rw_mutex);  
} while (true);
```

Readers-Writers Problem (Cont.)

- The structure of a reader process

```
do {
    wait(mutex);
    read_count++;
    if (read_count == 1)
        wait(rw_mutex);
    signal(mutex);

    /* ...
     * reading is performed */
    ...

    wait(mutex);
    read_count--;
    if (read_count == 0)
        signal(rw_mutex);
    signal(mutex);
} while (true);
```

Dining-Philosophers Problem



- Philosophers spend their lives alternating thinking and eating
- All philosophers pick up their left fork at the same time.
- Each philosopher waits for the right fork, which is already held by their neighbor.
- No philosopher can eat

Dining-Philosophers Problem Algorithm

- The structure of Philosopher i:

```
do {  
    wait (chopstick[i] );  
    wait (chopStick[ (i + 1) % 5] );  
  
    // eat  
  
    signal (chopstick[i] );  
    signal (chopstick[ (i + 1) % 5] );  
  
    // think  
  
} while (TRUE);
```

Problems with Semaphores

- Incorrect use of semaphore operations:
 - `signal(mutex) ...wait(mutex)`
 - `wait(mutex) ...wait(mutex)`
 - Omitting of `wait(mutex)` or `signal(mutex)` (or both)
- Deadlock and starvation are possible.

Monitors

- A high-level abstraction that provides a convenient and effective mechanism for process synchronization
- Abstract data type, internal variables only accessible by code within the procedure
- Only one process may be active within the monitor at a time
- But not powerful enough to model some synchronization schemes

```
monitor monitor-name
{
    // shared variable declarations
    procedure P1 (...) { .... }

    procedure Pn (...) {.....}

    Initialization code (...) { ... }
}
}
```

End of Lecture 7

Weekly Plan

- **Practical:** Shared Memory implementation in C
- **Workshop:**
 - Summary – Process Synchronization in OS
 - Discussion of Tutorial 7

Thank You